COMPTIA PENTEST+ PT0-003 · DOMAIN 3.0

Domain 3.2 — Analyze Output from Reconnaissance, Scanning & Enumeration

Turning raw scan / recon / enumeration output into validated, exploitable findings. Sort real from noise, judge coverage, fix a broken scan, and pick the right exploit.

> **Objective 3.2 —** Given a scenario, analyze output from reconnaissance, scanning, and enumeration phases.

MAP The analysis pipeline & the confusion matrix

Every result is a claim. The matrix classifies it against reality; the four-stage pipeline is the order you work in — classify, judge completeness, troubleshoot the config, then select and confirm an exploit.

SCANNER SAYS \ REALITY	vuln PRESENT	vuln ABSENT
POSITIVE (flagged)	**TP** ✓ True Positive — validate & exploit	**FP** ✗ False Positive — cull — wastes trust
NEGATIVE (reported clean)	**FN** ⚠ False Negative — MOST dangerous	**TN** ✓ True Negative — trust only if complete

TP flagged & real **FP** flagged, not real **FN** missed, real — worst **TN** clean & really clean

1. **CLASSIFY** — Is this finding real? Sort every result into the confusion matrix. →
2. **COMPLETENESS** — Did I actually see everything? Judge coverage before trusting 'clean'. →
3. **TROUBLESHOOT** — Why is the output wrong or empty? Fix the scan and re-run. →

4

SELECT & CONFIRM

Weaponize the confirmed finding: pick an exploit, prove it with a script.

1 Concept cards

Color-railed by pipeline stage. Exam tells are woven into each card, not bolted on at the end.

The validation confusion matrix

MENTAL MODEL

Every scan result is a claim that gets checked against reality. Two axes: what the scanner SAYS (positive = it flagged a vuln, negative = it reported clean) versus what is actually TRUE (the vuln is present or absent). The four cells are TP, FP, FN, TN.

3.2 is the whole discipline of moving a raw result out of the scanner's claim and into a validated cell. You never report a scanner's word as fact — you classify it first.

- Axes: Scanner says (Positive/Negative) x Reality (Present/Absent).
- Positive = flagged. Negative = reported clean. This is scanner-speak, not good/bad.
- Validation = manual re-test + scripting to confirm which cell a result really lands in.

EXAM TELL If a question describes 'the scanner reported X but...', the '...' tells you which cell it truly belongs to. The setup is always scanner-claim vs. ground-truth.

True positive (TP)

FLAGGED + REAL

The scanner flagged a vulnerability and it genuinely exists. This is the finding you validate, document, and carry into the attack phase. The 'good' outcome — the tool was right.

- Scanner: POSITIVE. Reality: PRESENT.
- Action: confirm, then prioritize (CVSS/EPSS) and select an exploit.
- Still validate it — a real vuln can be mis-scoped or already-mitigated in context.

EXAM TELL 'Nessus flagged CVE-2021-44228 and manual testing confirmed the host is exploitable' = TRUE POSITIVE. Watch for the confirming clause ('confirmed', 'verified', 'reproduced').

False positive (FP)

FLAGGED, NOT REAL

The scanner flagged something that isn't actually a vulnerability — a phantom. Common causes: banner/version-only detection when the patch was back-ported, a signature matching a non-exploitable code path, or a service that looks vulnerable but is compensated by other controls.

FPs waste tester and client time and, if they reach the report, destroy your credibility. Culling them by manual validation is a core 3.2 task.

- Scanner: POSITIVE. Reality: ABSENT.
- Classic cause: version-banner match without checking the actual patched behavior.
- Cost = wasted effort + lost trust. Not dangerous to the client, dangerous to your report.

EXAM TELL 'The scan reported the server as vulnerable, but the vendor back-ported the fix / the flagged CVE does not apply to this build' = FALSE POSITIVE. Keyword: 'does not actually exist'.

False negative (FN)

MISSED + REAL

The scanner reported clean, but a real vulnerability is present and it was missed. In security this is the MOST dangerous outcome: the tool gave you false confidence and a genuine hole goes unreported — an attacker's way in you failed to flag.

FNs are usually a coverage problem: unauthenticated scan couldn't see installed-software versions, ports outside the scanned range, a WAF filtered the probe, or the plugin feed was out of date and didn't know the CVE yet.

- Scanner: NEGATIVE. Reality: PRESENT.
- Most dangerous for a pentest — you leave the client exposed and don't know it.
- Drivers: unauthenticated scans, incomplete coverage, WAF/IPS filtering, stale feeds.

EXAM TELL 'Manual testing found an SQLi the automated scan never reported' = FALSE NEGATIVE. If the stem asks which error is WORST for a penetration test, the answer is false negative.

True negative (TN)

CLEAN + REALLY CLEAN

The scanner reported clean and the target really is clean. The quiet, correct baseline cell. Rarely the answer by itself, but it completes the matrix and is the outcome you can only TRUST once you've confirmed scan completeness.

- Scanner: NEGATIVE. Reality: ABSENT.
- Only trustworthy if coverage was complete — otherwise a 'clean' result may be an FN.
- This is why completeness is a separate stage: it turns 'negative' into a real TN.

EXAM TELL A 'clean' result is only a true negative if you can defend the coverage behind it. Exam framing links TN to scan-completeness: no completeness, no trustworthy negative.

Scan completeness

COVERAGE CHECK

Before you trust any 'negative', ask what the scan actually covered. Default Nmap scans the top 1000 TCP ports, not all 65535; UDP is frequently skipped because it's slow; hosts that block ping look 'down'; scans time out, get rate-limited, or abort partway.

Every gap in coverage is a false-negative factory. Completeness is the difference between 'nothing there' and 'I didn't look there.'

- Nmap default = top 1000 TCP ports. Use -p- for all 65535; -sU for UDP.
- Host looks down but is up -> it drops ping. Use -Pn to skip host discovery.
- Incomplete coverage is the #1 source of false negatives. Check ranges, hosts, protocols.

EXAM TELL 'The report shows no findings on a host you know runs a database' -> suspect an incomplete scan (port range / -Pn / UDP), not a clean host. Completeness gaps = hidden FNs.

Authenticated vs unauthenticated coverage

DEPTH DRIVER

Whether a scan had credentials changes how much it could possibly see. An unauthenticated scan sees the outside only (open ports, banners), so it misses installed-software patch levels, local config flaws, and missing OS patches — a rich source of false negatives.

Authenticated scans log in and enumerate installed packages and patch state, giving far fewer false negatives and more accurate results. (Detailed under 3.1; here it's the lever you check when a 'clean' result looks too clean.)

- Authenticated = deeper, fewer FNs, sees patch levels/local config.
- Unauthenticated = surface only, more FNs, easy to over-trust a 'clean' result.
- When completeness is doubted and creds exist, re-run authenticated.

EXAM TELL 'Unauthenticated scan reported the host as clean' is a completeness red flag: the depth wasn't there to rule out local/patch vulns. Pair this with false-negative reasoning.

Troubleshooting scan configurations

FIX & RE-RUN

When output is empty, wrong, or all 'filtered', the scan config — not the target — is usually the problem. Work a short checklist: is the host dropping ping (need -Pn)? Is a firewall/IPS eating the probes (everything 'filtered')? Did credentialed auth fail (wrong/expired creds, too-low privilege)? Is the plugin/NVT feed stale (missing recent CVEs)? Is timing too aggressive (-T5 drops packets)? Is the target/scope typed correctly?

Fixing the config and re-running is how you convert a misleading scan into a trustworthy one — and how you tell a real 'clean' from a broken run.

- Host 'down' but up -> firewall drops ICMP; add -Pn.
- All ports 'filtered' -> firewall/WAF/IPS between you and target.
- Empty credentialed results -> auth failed or account lacks privilege.
- Missing newer CVEs -> update Nessus plugins / OpenVAS NVT feeds.
- -T5 too aggressive -> dropped packets, unreliable; dial timing back.
- -sS (SYN) needs root; without it Nmap falls back to noisier -sT connect scan.

EXAM TELL A stem giving you a symptom ('host shows down', 'all filtered', 'no results with creds') wants the specific config fix. Map symptom -> flag/feed/privilege fix, then re-run.

Public exploit selection

MATCH THE TARGET

Once a finding is a validated true positive, you pick a public exploit to weaponize it. The #1 rule is EXACT MATCH: the exploit must fit the target's precise software version, platform, and architecture (an exploit for Apache 2.4.49 will not work on 2.4.50; a 32-bit payload won't fit a 64-bit target).

Sources: Exploit-DB (searchsploit), Metasploit modules, GitHub PoCs, PacketStorm, vendor advisories, mapped from the CVE. Vet before you fire: public PoCs can be booby-trapped or destructive, and some exploits crash the service (a DoS risk that may violate the rules of engagement on fragile production systems).

- Match version + platform + architecture EXACTLY — the top trap on this objective.
- searchsploit / Exploit-DB to find; Metasploit to run (module rank = reliability).
- Read public exploit code before running — it may be malicious or destructive.
- Check stability: will it crash the target? A DoS may breach the RoE.
- PoC proves the flaw; a weaponized/functional exploit gives access.

EXAM TELL 'Which exploit for <product> <version>?' -> pick the one whose version/platform matches exactly. Version mismatch and 'run untrusted PoC blindly' are the planted wrong answers.

Descriptors that drive selection (CVSS/CVE/CWE/EPSS)

PRIORITIZE

To decide WHICH validated finding to weaponize first, use the standard descriptors. CVE identifies the specific flaw; CWE names the weakness class; CVSS base score rates severity; EPSS estimates the probability it will be exploited in the wild.

For analysis you often combine them: CVSS tells you how bad, EPSS tells you how likely. (These are detailed under objective 4.1 — here they're the ranking lens you apply to confirmed 3.2 results before exploit selection.)

- CVE = the specific vulnerability ID. CWE = the weakness category.
- CVSS = severity (how bad). EPSS = likelihood of real-world exploitation (how likely).
- High CVSS + high EPSS = weaponize first. Cross-referenced fully in 4.1.

EXAM TELL If asked to prioritize among confirmed vulns, remember CVSS = severity, EPSS = likelihood. Don't confuse the two — that swap is a favorite distractor.

Use scripting to validate results

PROVE IT INDEPENDENTLY

Scripting gives you an independent confirmation that a finding is real, cutting false positives, and lets you write a custom check the scanner never ran, catching false negatives. Typical moves: a curl request to check a header or response, a Python socket banner-grab to confirm a version, a bash loop to test the same condition across many hosts, an Nmap NSE script, or searchsploit to map a version to known exploits.

The point is verification at your own hand — you don't take the scanner's word, you reproduce the result with a small script (bash / Python / PowerShell) and see it yourself.

- Scripted re-test confirms TPs and culls FPs (independent second opinion).
- Custom checks catch what the scanner missed (targets FNs).
- Tools of the trade: curl, Python sockets/requests, bash loops, Nmap NSE, searchsploit.
- Ties to 2.3 / 4.10 scripting — read/modify a snippet to validate a finding.

EXAM TELL 'Confirm the finding before reporting' or a short curl/Python snippet checking one host = scripting-to-validate. The exam wants independent reproduction, not re-trusting the tool.

2 Critical comparisons

The blurred pairs this objective loves to test. Read the verdict line last — it's the tie-breaker.

False positive vs False negative

False positive

Scanner FLAGGED it, but it isn't real. Cost = wasted time + damaged credibility. Safe for the client — nothing is actually exposed.

vs

False negative

Scanner MISSED a real vuln. Cost = the client stays exposed and you don't report it. The dangerous one for a pentest.

▸ Worst-for-a-pentest = FALSE NEGATIVE (a real hole goes unflagged). Most-embarrassing-in-a-report = FALSE POSITIVE.

True positive vs False positive

True positive

Flagged AND real. Validate, then exploit. The tool was right.

vs

False positive

Flagged but NOT real. Validate, then discard. The tool was wrong.

▸ Both are POSITIVE (flagged). The difference is only whether reality backs it up — which is exactly what validation decides.

Discovery (3.1) vs Validation (3.2)

Discovery — 3.1

RUN the scans/recon/enum. Produce raw output. 'Here is everything the tools found.'

vs

Validation — 3.2

ANALYZE that output. Sort real from noise, judge coverage, pick exploits. 'Here is what's actually true and actionable.'

▸ 3.1 generates the pile; 3.2 is the sieve. If the stem is about trusting/confirming/triaging output, it's 3.2.

Scan completeness vs Scan accuracy

Completeness (coverage)

Did I look everywhere? Ports, hosts, protocols, auth depth. Gaps -> false negatives.

vs

Accuracy (correctness)

Was what I found real? Signal vs noise. Errors -> false positives.

▸ Completeness fights false negatives; accuracy fights false positives. Two different failure modes, two different fixes.

Proof of concept vs Weaponized exploit

PoC

Demonstrates the flaw exists — may just crash it or pop a calc. Proves, doesn't necessarily pwn.

vs

Weaponized / functional

Reliable, delivers a payload/shell. Ready for real use against the target.

▸ A PoC confirms; a weaponized exploit gets you access. Selection may need you to develop a PoC into something usable.

CVSS vs EPSS

CVSS
Base score = how SEVERE the vuln is (impact + exploitability characteristics). Static-ish.

vs

EPSS
Probability it will be EXPLOITED in the wild soon. Threat-driven, changes over time.

▸ Severity (CVSS) is not likelihood (EPSS). Prioritize confirmed findings by BOTH; never swap their meanings.

searchsploit / Exploit-DB vs Metasploit

searchsploit / Exploit-DB
FIND public exploits and PoCs; search offline by product/version. A catalog.

vs

Metasploit
RUN exploits as modules with payloads; module RANK signals reliability. A launcher.

▸ One is for locating the right exploit, the other for firing it. 'Find the exploit for version X' = searchsploit.

3 Scenario decoder

Stem clue → the concept it points to. These phrasings recur almost verbatim.

Scanner flagged the host as vulnerable, but the vendor back-ported the patch so it isn't	→	CLASSIFY **False positive**
Manual testing found an injection flaw the automated scan never reported	→	CLASSIFY **False negative**
Nessus flagged a CVE and you reproduced exploitation by hand	→	CLASSIFY **True positive**
Which single error is worst for a penetration test?	→	CLASSIFY **False negative (real hole left unreported)**
A 'clean' result you can actually trust because coverage was full	→	CLASSIFY **True negative**
Host is definitely up but the scan reports it as down / no ports	→	TROUBLESHOOT **Firewall drops ping — re-run with -Pn**
Every port comes back 'filtered'	→	TROUBLESHOOT **Firewall / WAF / IPS between you and target**
Credentialed scan returns almost nothing	→	TROUBLESHOOT **Auth failed or account lacks privilege — fix creds**
Scanner isn't reporting a CVE published last week	→	TROUBLESHOOT **Stale plugin/NVT feed — update it and re-scan**

Scan only checked 1000 ports on a box you know runs a DB on a high port	→	COMPLETENESS **Incomplete coverage — use -p- (and -sU for UDP)**
Unauthenticated scan says the host is clean of missing patches	→	COMPLETENESS **Depth gap — re-run authenticated to rule out FNs**
Need to be sure a finding is real before it goes in the report	→	SELECT & CONFIRM **Validate independently (manual re-test / scripting)**
Pick an exploit for Apache httpd 2.4.49 specifically	→	SELECT & CONFIRM **Match version+platform exactly (2.4.50 module won't work)**
A public GitHub PoC — should you just run it?	→	SELECT & CONFIRM **Read/vet the code first; it may be malicious or crash the target**
Rank two confirmed vulns: one high CVSS, one high EPSS	→	SELECT & CONFIRM **Use both — CVSS=severity, EPSS=likelihood; ideally weaponize high/high**
Write a quick curl/Python check to confirm the flagged header is present	→	SELECT & CONFIRM **Use scripting to validate results**

4 Practice quiz

Fourteen exam-style items. Answer in your head, then reveal the key and the reasoning — the explanation is the point, not the score.

Q1 A vulnerability scanner reports that a web server is affected by CVE-2023-XXXX. On manual inspection, the patch has been applied and the flaw cannot be triggered. How do you classify this?

- A True positive
- B False positive
- C False negative
- D True negative

ANSWER **B · False positive**

The scanner flagged (positive) a vuln that isn't actually present (reality = absent) = false positive. Often caused by version/banner detection missing a back-ported fix.

Q2 During manual testing you discover a SQL injection that the automated scan never flagged. This is an example of a:

A False positive

B True positive

C False negative

D Scan completeness error only

ANSWER **C · False negative**

A real vuln (present) that the scanner reported clean (negative) = false negative — the most dangerous outcome because the exposure would otherwise go unreported.

Q3 Which result type is generally the MOST dangerous in the context of a penetration test?

A False positive

B True positive

C True negative

D False negative

ANSWER **D · False negative**

A false negative leaves a genuine vulnerability unflagged, giving false assurance. False positives waste time and hurt credibility but don't leave the client exposed.

Q4 A host you know is online shows as 'down' with no open ports in the Nmap output. The BEST next step is:

A Report the host as clean

B Re-run with -Pn to skip host discovery

C Switch to a UDP scan only

D Lower the CVSS score

ANSWER **B · Re-run with -Pn to skip host discovery**

The host is almost certainly dropping ICMP/ping, so Nmap's host discovery marks it down. -Pn skips discovery and scans it anyway. Reporting it clean would be a false negative.

Q5 Every port on a target returns the state 'filtered'. This most likely indicates:

- A The target is fully patched
- B A firewall/WAF/IPS is dropping your probes
- C You need authenticated credentials
- D The scan is complete and accurate

ANSWER **B · A firewall/WAF/IPS is dropping your probes**

'Filtered' means probes are being dropped with no clear response — the signature of a firewall/IPS/WAF between you and the target, i.e., a scan-configuration/troubleshooting issue.

Q6 A default Nmap scan of a server returns no database service, but you know it hosts one on port 5432. The most likely cause is:

- A A false positive
- B The database is patched
- C Incomplete coverage — default scans the top 1000 ports
- D EPSS filtered the result

ANSWER **C · Incomplete coverage — default scans the top 1000 ports**

Nmap's default scans only the top 1000 TCP ports. A service on 5432 can fall outside that set. Use -p- for all 65535 ports. This is a scan-completeness gap producing a false negative.

Q7 You have confirmed a true positive on 'Apache httpd 2.4.49'. When selecting a public exploit, the single most important check is:

- A That it has the most GitHub stars
- **B That it matches the exact version and platform**
- C That it is written in Python
- D That its CVSS is above 9.0

ANSWER **B · That it matches the exact version and platform**

Exploits are version- and platform-specific; a module for 2.4.50 (or the wrong architecture) will fail against 2.4.49. Exact match is the top trap on this objective.

Q8 Before running an unfamiliar public proof-of-concept exploit from a GitHub repo against a client system, you should FIRST:

- A Run it immediately to save time
- **B Read/vet the code for malicious or destructive behavior**
- C Increase scan timing to -T5
- D Delete your engagement notes

ANSWER **B · Read/vet the code for malicious or destructive behavior**

Public PoCs can be booby-trapped or can crash the service (a DoS that may violate the rules of engagement). Reviewing the code before execution is basic due diligence.

Q9 Two validated findings compete for attention: Finding A has a high CVSS base score; Finding B has a high EPSS score. What do these tell you?

A Both mean the same thing

B A is more likely to be exploited; B is more severe

C A is more severe; B is more likely to be exploited in the wild

D Neither is relevant to selection

ANSWER **C · A is more severe; B is more likely to be exploited in the wild**

CVSS measures severity (how bad), EPSS measures the probability of real-world exploitation (how likely). They're complementary; ideally you prioritize high-severity AND high-likelihood.

Q10 A credentialed vulnerability scan returns almost no findings, though the OS is old. The FIRST thing to check is:

A Whether the authentication actually succeeded / has sufficient privilege

B Whether EPSS is enabled

C Whether the target is a honeypot

D Whether to lower -T timing

ANSWER **A · Whether the authentication actually succeeded / has sufficient privilege**

Sparse results from a credentialed scan usually mean auth failed or the account lacks privilege, so the scan ran effectively unauthenticated. Fix the credentials/permissions and re-run.

Q11 Which action best represents 'use scripting to validate results'?

A Re-reading the scanner's PDF report

B Writing a short curl/Python check that independently reproduces the flagged condition

C Raising the finding's CVSS

D Turning off the WAF

ANSWER **B · Writing a short curl/Python check that independently reproduces the flagged condition**

Scripting-to-validate means independently confirming the finding at your own hand — e.g., a small curl or Python script that reproduces the result — rather than re-trusting the tool's output.

Q12 A vulnerability feed hasn't been updated in months and the scanner is missing recently disclosed CVEs. This condition primarily causes:

A False positives

B False negatives

C True positives

D Better scan completeness

ANSWER **B · False negatives**

An out-of-date plugin/NVT feed can't detect CVEs it doesn't know about, so real vulns go unreported = false negatives. Fix: update the feed and re-scan.

Q13 You want to trust a 'no vulnerabilities found' result on a critical host. Before you do, you must establish:

A That the exploit worked

B Scan completeness (coverage of ports, hosts, protocols, auth depth)

C A higher EPSS

D That it was a stealth scan

ANSWER **B · Scan completeness (coverage of ports, hosts, protocols, auth depth)**

A negative is only a true negative if coverage was complete. Unscanned ports, skipped UDP, or an unauthenticated run can hide real vulns, turning a 'clean' result into a false negative.

Q14 You need to locate a public exploit for a specific product version offline. The most appropriate tool is:

A Metasploit's db_nmap

B searchsploit (Exploit-DB)

C Wireshark

D Mimikatz

ANSWER **B · searchsploit (Exploit-DB)**

searchsploit queries a local copy of Exploit-DB by product/version — ideal for FINDING the right public exploit. Metasploit is for running modules once you've chosen one.

5 Keyword table

Every term dotted to its pipeline stage. Use the search box up top to filter.

True positive (TP)	Flagged AND real — validate and exploit.
False positive (FP)	Flagged but NOT real — validate and discard; wastes time, hurts credibility.
False negative (FN)	Missed but real — most dangerous; leaves client exposed.
True negative (TN)	Clean and really clean — only trustworthy with full coverage.
Confusion matrix	Scanner-says (pos/neg) x reality (present/absent) = TP/FP/FN/TN.
Validation	Manual re-test + scripting to confirm which cell a result belongs in.
Scan completeness	Did coverage include all ports/hosts/protocols/auth depth? Gaps = FNs.
Top 1000 ports	Nmap default TCP scan scope; use -p- for all 65535.
-Pn	Skip host discovery; scan a host that drops ping (looks 'down').
Filtered	Probes dropped with no response — firewall/WAF/IPS in the path.
Authenticated scan	Logs in; sees patch levels/local config; far fewer false negatives.
Unauthenticated scan	Surface only; misses local/patch vulns; more false negatives.
Troubleshoot scan config	Fix -Pn/creds/feed/timing/scope, then re-run to trust the output.
Stale plugin / NVT feed	Out-of-date detections miss new CVEs -> false negatives.

-T5 timing	Too aggressive; drops packets and reduces reliability — dial back.
-sS vs -sT	SYN scan needs root; falls back to noisier connect scan without it.
Public exploit selection	Match target version+platform+arch EXACTLY; vet the code first.
searchsploit / Exploit-DB	Find public exploits/PoCs by product and version.
Metasploit module rank	Reliability rating (excellent...low) of an exploit module.
PoC vs weaponized	PoC proves the flaw; weaponized/functional delivers a payload/shell.
CVSS	Severity of a vuln (how bad).
EPSS	Probability of real-world exploitation (how likely).
CVE / CWE	CVE = specific vuln ID; CWE = weakness class/category.
Scripting to validate	Independently reproduce a finding (curl/Python/bash/NSE) to confirm TP / catch FN.
DoS risk (exploit)	Some exploits crash the service — may breach rules of engagement.

6 Exam strategy — the night-before strip

Read every 3.2 stem as scanner-claim vs. ground-truth. The verb in the confirming clause ('confirmed', 'does not actually exist', 'manual testing found') tells you the true cell.

If asked for the WORST error in a pentest, it's the false negative — a real hole left unreported.

A 'clean'/negative result is only trustworthy after you've defended completeness. No coverage, no true negative.

Symptom -> config fix: down-but-up = -Pn; all-filtered = firewall/IPS; empty-credentialed = fix auth; missing new CVEs = update the feed; unreliable = dial timing back.

Exploit selection = EXACT version + platform + architecture match, and vet/READ public code before firing.

Keep CVSS (severity) and EPSS (likelihood) straight — swapping them is a planted distractor.

'Confirm before reporting' points to independent validation — manual re-test or a small script, not re-trusting the tool.

Built to the official PT0-003 v3.0 objective 3.2 sub-bullets. Not affiliated with or endorsed by CompTIA.